Tentamen TEN1 HI1029 2022-03-17

Sista ordinarie insläpp: 8.00 – dörren stängs.
Tentavakterna delar ut konton. Logga in i tentasystemet så fort som möjligt.
När alla loggat in (förhoppningsvis 8.15) börjar tentamen och pågår i 4h 45min (tex 8.15-13.00).
8.30 Insläpp för försenade.

Extra skrivtid 50%: 7h 10min (tex 8.15-15.25) *(endast 1h extra skrivtid för A eller B)*
Skrivvakter: Anteckna start- och inlämningstid.

Hjälpmedel: papper (tomma), penna

Logga in med tentamenskontot ni får av skrivvakten. Logga inte ut förrän ni är klara med hela tentan.

Svaren på teorifrågorna (fråga ett och två) skall besvaras i en textfil döpt till teori.txt som ni lägger direkt under H. Svaren på programmeringsfrågorna (tre till sju) ska vara projekt som sparas på H.

Eclipse: Välj H som ert workspace i eclipse. Skapa ett nytt projekt: **Project Name:** TEN1Eclipse.
Netbeans: Skapa ett nytt Java Application Project: **Project Name:** TEN1NetBeans, **Project Location:** H.
IntelliJ: Skapa ett nytt projekt: **Project Name:** TEN1IntelliJ, **Project Location:** H.

Alla: Skapa ett package per uppgift och kalla dessa uppgift3, uppgift4,…, uppgift7. Kopiera hjälpfiler från P/HI1029/ till rätt package.

Även om ni inte löser hela uppgiften så kan ni få delpoäng men ni måste se till att koden kompilerar och kan köras. Både output och koden bedöms och det är viktigt att denna är välskriven och uppfyller uppgiftens instruktioner. Om ett program är ineffektivt kan detta ge poängavdrag. Det är meningen att man skall använda de tekniker vi lärt oss där detta är fördelaktigt. Välj bra namn för variabler, metoder och klasser. Se till att programmet har en bra struktur. Ni behöver på denna tentamen inte kommentera.

Hjälpfiler hittar ni i P/HI1029/ och Java API i P/Java/api/index.html.

Del 1 (15p): För godkänt krävs minst 1 poäng på teorifrågorna samt 9 poäng totalt. Om man får minst en poäng på teorifrågorna får man betyg enligt: 9p – E, 10p – D, 12p – C. Om man inte får minst en poäng på teorifrågorna men minst 8,5 poäng totalt får man Fx.

Del2: Bedöms bara om man får betyg C på del 1 och bedöms då med A, B eller C. Observera att om du har extra skrivtid får du bara använda 1 extra timme för att kunna få betyg A eller B.

Examinator och rättande lärare: Nicklas Brandefelt

Del 1

1. Quicksort har ett worst-case som är $O(n^2)$. Hur uppkommer det? Förklara vad som händer och varför det blir $O(n^2)$. (1p)
   Svaret skall lämnas i en textfil, teori.txt i H.

2. Beskriv hur öppen adressering hanterar krockar i en hashtabell. Beskriv kortfattat hur det fungerar för put, get och remove. (1p)
   Svaret skall lämnas i en textfil, teori.txt i H.

3. ***Ni som har fått godkänt på krysstalen i år hoppar över denna uppgift.***
   I katalogen: P/HI1029/uppgift3 hittar du vår enkellänkade lista från föreläsningen (men utan medlemmen size). Där finns också filen Main.java som ska fungera att köra när du är klar. Du får i denna uppgift inte lägga till några medlemsvariabler eller ändra andra metoder i listan. Däremot får du som alltid gärna skriva privata hjälpmetoder om du anser att det behövs. Kom ihåg att det ska gå att anropa metoderna också på en tom lista.

   a) Skriv metoden remove till listan. (2p)

   b) Skriv metoden size till listan. Använd rekursion för att lösa uppgiften. (1p)

4. I denna uppgift börjar man längst upp till vänster i en matris av heltal. Man ska ta sig längst ner till höger. I varje steg kan man hoppa ett eller två steg åt höger eller ett steg nedåt. Observera att man måste sluta på rutan längst ned till höger. Målet är att hitta den väg som ger största summan om man summerar alla tal man passerat. Observera att man inte summerar tal man hoppar över. I exemplet nedan är vägen med störst summa markerad.

2	-1	1	2	1	3
4	4	-3	5	2	2
1	2	-2	3	3	1
5	-4	-3	1	4	4
0	2	3	2	-5	1

   a) Skriv en metod som tar en int-matris som indata och returnerar största möjliga summa enlig ovan. I exemplet ovan ska den alltså returnera 30. Lös uppgiften med hjälp av rekursion och djupet först. (3p)

   b) Effektivisera din lösning med hjälp av dynamisk programmering (tabulering). Om du är osäker på denna kan du spara en version av metoden utan och en med tabulering i filen. (2p)

   Ett enkelt skal hittar ni i P/HI1029/uppgift4.

5. I den här uppgiften utgår vi ifrån vårt binära sökträd från föreläsningen (förenklat). Utgå ifrån filerna i P/HI1029/uppgift5. Kom ihåg att metoderna ska kunna hantera även ett tomt träd. Du ska inte lägga till några medlemsvariabler men får använda privata hjälpmetoder.

   a) Skriv toString – metoden. Den ska returnera en sträng med datat i trädet i storleksordning, minst till störst. (1p)

   b) Skriv metoden isBalanced. Denna ska returnera true om trädet är balanserat och false om trädet inte är balanserat. Du behöver inte optimera effektiviteten i denna uppgift. (2p)

   Tips: I ett balanserat träd skiljer sig höjden för vänster och höger underträd till varje nod med maximalt 1.

6. I ett lager har man kartonger som klarar att packas med en viss vikt (i hela kg). Man har också varor med olika vikter (också i hela kg). Av varje sorts vara (med en specifik vikt) finns det obegränsat antal. Det finns alltid en sorts vara som har vikten 1 kg. Målsättningen är att fylla kartongen helt full med så få varor som möjligt. Du får full poäng om du löser problemet med en algoritm som alltid hittar den optimala lösningen eller om du löser problemet med en bra girig algoritm som ibland inte hittar den optimala lösningen.

   Skriv en metod som tar kartongens kapacitet och en array med vikterna för de olika varorna och returnerar hur många varor som behövs för att fylla kartongen. (2p)

   Exempel:

   Kartongens kapacitet: 10 kg. Varornas vikter: 2kg, 4kg, 1kg. Svaret är 3 varor eftersom vi kan fylla kartongen med 2st 4kg-varor och en 2kg-vara. Detta ska även en girig algoritm klara.

   Kartongens kapacitet: 20 kg. Varornas vikter: 2kg, 15kg, 7kg, 6kg, 1kg. Svaret är 3 varor eftersom vi kan fylla kartongen med 2st 7kg-varor och en 6kg-vara. Här är det okej om en girig algoritm svarar 4 varor.

   I katalogen P/HI1029/uppgift6 hittar du ett enkelt skal.

Vänd blad!

Del 2

7. En robot är konstruerat så att när den har börjat åka åt ett håll så fortsätter den till dess att den når en vägg. När den når en vägg kan den igen välja att åka åt valfritt håll. Ditt jobb är att skriva en metod som givet ett rum med väggar, en startposition och ett mål beräknar det kortaste antalet sträckor roboten behöver för att nå målet. Observera att det inte räcker att passera målet. Den måste stanna på målet. Du kan räkna med att rummet har väggar runt hela kanten. Skriv en metod som tar en char-matris med X för vägg, S för start och M för mål och returnerar minsta antalet sträckor roboten behöver för att nå målet. Om det är omöjligt att nå till målet ska metoden returnera -1. Försök göra metoden så effektiv som möjligt.

```
XXXXXXXXXXXXXXXXXXXXXX
X                    X
X       X            X
X  S   XXX     XX     X
X              MX    X
X                    X
X                    X
X       X     XX     X
X       X            X
X       X          XXX
X             X      X
X             X      X
X                    X
XXXXXXXXXXXXXXXXXXXXXX
```

I exemplet ovan så ska metoden svara 6.

```
XXXXXXXXXXXXXXXXXXXXXX
X                 X  X
X       X            X
X  S   XXX     XX     X
X              MX    X
X                    X
X                    X
X       X     XX     X
X       X            X
X       X          XXX
X             X      X
X             X      X
X                    X
XXXXXXXXXXXXXXXXXXXXXX
```

Och för denna blir svaret 8.

Ett enkelt skal hittar ni i P/HI1029/uppgift7.